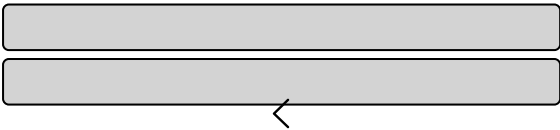


[View in English](#)



- [Apple Developer](#)

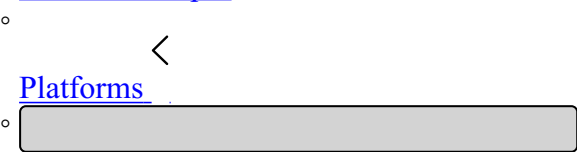


Explore Get Started

- [Overview](#)
- [Learn](#)
- [Apple Developer Program](#)

Stay Updated

- [Latest News](#)
- [Hello Developer](#)



Explore Platforms

- [Apple Platforms](#)
- [iOS](#)
- [iPadOS](#)
- [macOS](#)
- [tvOS](#)

- [visionOS](#)
- [watchOS](#)
- [App Store](#)

Featured

- [Design](#)
- [Distribution](#)
- [Games](#)
- [Accessories](#)
- [Web](#)
- [Home](#)
- [CarPlay](#)

- [Technologies](#)



Explore Technologies

- [Overview](#)
- [Xcode](#)
- [Swift](#)
- [SwiftUI](#)

Featured

- [Accessibility](#)
- [App Intents](#)
- [Apple Intelligence](#)
- [Games](#)
- [Machine Learning & AI](#)
- [Security](#)
- [Xcode Cloud](#)

- [Community](#)

- 



Explore Community

- [Overview](#)
- [Meet with Apple events](#)
- [Community-driven events](#)
- [Developer Forums](#)
- [Open Source](#)

Featured

- [WWDC](#)
- [Swift Student Challenge](#)
- [Developer Stories](#)
- [App Store Awards](#)
- [Apple Design Awards](#)
- [Apple Developer Centers](#)

- 
[Documentation](#)

- 



Explore Documentation

- [Documentation Library](#)
- [Technology Overviews](#)
- [Sample Code](#)
- [Human Interface Guidelines](#)
- [Videos](#)

Release Notes

- [Featured Updates](#)
- [iOS](#)

- [iPadOS](#)
- [macOS](#)
- [watchOS](#)
- [visionOS](#)
- [tvOS](#)
- [Xcode](#)

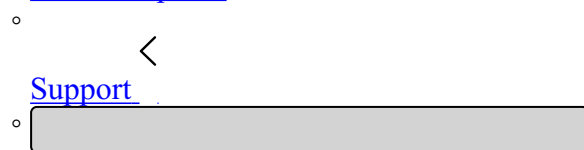


Explore Downloads

- [All Downloads](#)
- [Operating Systems](#)
- [Applications](#)
- [Design Resources](#)

Featured

- [Xcode](#)
- [TestFlight](#)
- [Fonts](#)
- [SF Symbols](#)
- [Icon Composer](#)



Explore Support

- [Overview](#)
- [Help Guides](#)
- [Developer Forums](#)
- [Feedback Assistant](#)

- [Contact Us](#)

Featured

- [Account Help](#)
- [App Review Guidelines](#)
- [App Store Connect Help](#)
- [Upcoming Requirements](#)
- [Agreements and Guidelines](#)
- [System Status](#)

•



globalnav



Quick Links

- → → [Events](#)
- → → [News](#)
- → → [Forums](#)
- → → [Sample Code](#)
- → → [Videos](#)

•



=



[Videos](#)

[Open Menu](#) [Close Menu](#)

- [Collections](#)
- [Topics](#)
- [All Videos](#)
- [About](#)

[More Videos](#)



- [About](#)
- [Summary](#)
- [Transcript](#)

• Build a SwiftUI app with the new design

Explore the ways Liquid Glass transforms the look and feel of your app. Discover how this stunning new material enhances toolbars, controls, and app structures across platforms, providing delightful interactions and seamlessly integrating your app with the system. Learn how to adopt new APIs that can help you make the most of Liquid Glass.

Chapters

- 0:00 - [Introduction](#)
- 3:07 - [App structure](#)
- 7:49 - [Toolbars](#)
- 11:10 - [Search](#)
- 14:03 - [Controls](#)
- 17:57 - [Liquid Glass effects](#)
- 21:31 - [Next steps](#)

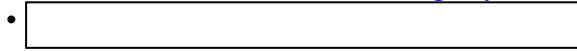
Resources

- [Applying Liquid Glass to custom views](#)
- [Landmarks: Building an app with Liquid Glass](#)
- [Adopting Liquid Glass](#)
 - [HD Video](#)
 - [SD Video](#)

Related Videos

WWDC25

- [Get to know the new design system](#)



Search this video...

Hi. I'm Franck, an engineer on the SwiftUI team. And in this video, you will learn how to build a great app with the new design. iOS 26 and macOS Tahoe introduce significant updates to the look and feel of apps and system experiences. At the heart of these updates is a brand new, adaptive material for controls and navigational elements that we call Liquid Glass. It takes inspiration from the optical properties of glass and the fluidity of liquid to create a lightweight, dynamic material that helps elevate the underlying content across various components.

As you scroll through content, the glass automatically adapts to the content underneath, changing from light to dark. With a new and refreshed look across all platforms, controls come alive during interaction. Controls like toggles, segmented pickers, and sliders now transform into liquid glass during interaction, creating a delightful experience! These updates apply across all the platforms that your app runs on. Watch “Meet Liquid Glass” for a deeper dive into the design of this new material.

Then, check out “Get to know the new design system” to learn best practices with the new design.

Sometimes, in life, to gain clarity and focus on what's truly important, you may need to re-invent yourself and look at the bigger picture. Today, I'm applying this wisdom to the Landmarks app, a sample project available on the Apple Developer website. I'll showcase elements of the new design system across Apple platforms and I'll bring even greater clarity and focus to the Landmarks app by adopting the new APIs. When you build your app with the Xcode 26 SDKs, you'll notice changes throughout the UI. I'll begin with updates to structural app components like TabView and NavigationSplitView. Then, I'll cover the new look and behavior of toolbars. After that, I'll share updates to the search experience that enhance consistency and ease of use. Then, I'll show how controls come to life with Liquid Glass! I'll finish by describing how you can adopt glass into your own custom UI elements.

In each of these five areas there are improvements you will get automatically and new APIs to customize the experience even further. App structure refers to the family of APIs that define how people navigate your app.

These include views and modifiers like NavigationSplitView, TabView and Sheets! Every one of these members is refined for the new design. NavigationSplitView allows navigating through a well-defined hierarchy of possibly many root categories.

They now have a Liquid Glass sidebar that floats above your content.

This breathtaking hero banner in Landmarks illustrates this, with the pink blossoms refracting against the sidebar. But the Landmarks team didn't travel to all these fantastic destinations, framing spectacular pictures only to have them appear clipped, no matter how gorgeous the sidebar is! With the new `backgroundExtensionEffect` modifier, views can extend outside the safe area, without clipping their content.

If I hide the sidebar for a moment, you'll see what's happening behind it.

The image is mirrored and blurred outside of the safe area, extending the artwork while leaving all its content visible. The new design makes inspectors shine, with more Liquid Glass! Opposite the sidebar in Landmarks, the inspector hosts content with a more subtle layering. This associates the inspector with its related selection.

TabViews provide persistent, top-level navigation. They provide an overview of possibilities at a glance and optimize for switching from section to section, maintaining context within each section.

With the new design, the tab bar on iPhone floats above the content, and can be configured to minimize on scroll.

This lets your app's content remain the star of the show.

To adopt this behavior, use the `tabBarMinimizeBehavior` modifier. In this example, the TV app uses the `onScrollDown` behavior.

With this configuration, the tab bar re-expands when scrolling in the opposite direction.

Now, suppose your app has additional controls that you want close at hand, like this playback view in Music.

Place a view above the bar with the `tabViewBottomAccessory` modifier. This takes advantage of the extra space provided by the tab bar's collapsing behavior.

Inside your accessory view, read the `tabViewBottomAccessoryPlacement` from the environment. Then, adjust the content of your accessory when it collapses into the tab bar area.

Alright, I showed you how `NavigationSplitView` is beautiful in Landmarks with the new design, and I shared ways you can adapt TabView-based apps too. Next, I'll present sheets! When creating a collection of landmarks, a sheet of landmark options gets presented. On iOS 26, partial height sheets are inset by default with a Liquid Glass background.

At smaller heights, the bottom edges pull in, nesting in the curved edges of the display.

When transitioning to a full height sheet, the glass background gradually transitions, becoming opaque and anchoring to the edge of the screen.

If you've used the `presentationBackground` modifier to apply a custom background to your sheets, consider removing that and let the new material shine.

Sheets can also directly morph out of buttons that present them. To have the presentation content morph out of the source view, make the presenting toolbar item a source for a navigation zoom transition. And mark the content of your sheet as the destination of the transition.

Like sheets, other presentations such as menus, alerts, and popovers flow smoothly out of liquid glass controls, drawing focus from their action to the presentation's content.

In the new design, dialogs also automatically morph out of the buttons that present them! With our app structure standing pretty strong, let's move onto toolbars! In the new design, toolbar items are placed on a Liquid Glass surface that floats above your app's content and automatically adapts to what's beneath it. Toolbar items are automatically

grouped. When I build the Landmarks app with Xcode 26, my custom toolbar items are grouped separately from the system-provided back button.

I want to emphasize that the “favorite” and “add to collection” buttons are related actions. So, I used the new `ToolbarSpacer` API with fixed spacings to split them into their own group.

This provides visual clarity that the grouped actions are related, while the separated actions, like the share link and inspector, have distinct behavior. `ToolbarSpacer` can also be used to create a flexible space that expands between toolbar items.

The Mail app uses this technique to create a leading-aligned filter item and a trailing-aligned group with the search and compose items.

Some toolbar items can do without this visual grouping, like this item from Books showing my avatar. Apply the `sharedBackgroundVisibility` modifier to separate an item into its own group without a background.

In the Landmarks app, I added a feature that allows friends to react to my landmarks collection. I would like to add an indicator on my notification item when there is a new reaction. I don’t want to miss that sweet, sweet external validation.

By using the badge modifier on toolbar items that sweet validation is just one line of code away! I applied the badge modifier to my toolbar item’s content to display this indicator.

In addition to grouping and badging items in toolbars, the new design introduces a few other changes. Icons use monochrome rendering in more places, including in toolbars.

The monochrome palette reduces visual noise, emphasizes your app’s content, and maintains legibility.

You can still tint icons with a tint modifier, but use this to convey meaning, like a call to action or next step, but not just for visual effect.

In the new design, an automatic scroll edge effect keeps controls legible.

It is a subtle blur and fade effect applied to content under system toolbars. If your app has any extra backgrounds or darkening effects behind the bar items, make sure to remove them, as these will interfere with the effect.

For denser UIs with a lot of floating elements, like in the calendar app, tune the sharpness of the effect on your content with the `scrollEdgeEffectStyle` modifier.

Turning from toolbars, next I’ll show how to craft canny search experiences with the new design. There are some big updates to two key patterns for search across all platforms.

Search in the toolbar places the field at the bottom of the screen, within easy reach.

And on iPad and Mac, it appears in the top-trailing position of the toolbar.

The second pattern is to treat it as a dedicated page in a multi-tab app. For the Landmarks app, I placed the search in the top trailing corner. When using this placement, you should make as much of your app’s content available through search.

The search field appears on its own Liquid Glass surface.

A tap activates it and shows the keyboard. To get this variant in Landmarks, I applied the searchable modifier on the `NavigationSplitView`. Declaring the modifier here indicates that search applies to the entire `NavigationSplitView`, not just one of the columns.

On iPhone, this variant automatically adapts to bring the search field at the bottom of the display.

Depending on device size, number of toolbar buttons, and other factors, the system may choose to minimize the search field into a toolbar button, like the one shown here in Mail.

When I tap on the button, a full-width search field appears above the keyboard.

If you want to explicitly opt-in to the minimized behavior, say because search isn't a main part of your app's experience, use the new `searchToolbarBehavior` modifier.

Searching in multi-tab apps is often done in a dedicated search page. The pattern is used by apps across all our platforms, such as the Health app to check my fitness trends.

To do this in your app, set a search role on one of your tabs and place a searchable modifier on your `TabView`.

When someone selects this tab, a search field takes the place of the tab bar, and the content of the tab is shown.

People can interact with your browsing suggestions, or tap on the search field to bring up the keyboard and continue with specific search terms.

On iPad and Mac, when someone selects the search tab, the search field appears centered above your app's browsing suggestions. These patterns give you flexibility and control over the search experience in your app! Speaking of control, next I'll turn to updates to standard controls. The new design creates a strong family resemblance across platforms for controls like buttons, sliders, menus, and more.

I'm going to start with updates to buttons, one of the most common controls. Bordered buttons now have a capsule shape by default, harmonious with the curved corners of the new design. Mini, small, and medium size controls on macOS retain a rounded-rectangle shape, which preserves horizontal density.

And the existing button border shape modifier enables you to specify the shape for any size.

Control heights are updated for the new design.

Most controls on macOS are slightly taller, providing a little more breathing room around the control label, and enhancing the size of the click targets.

For compatibility with existing high-density layouts, like complex inspectors and popovers, the existing `controlSize` modifier can be applied to a single control or across an entire set of controls. And for your most important, prominent actions there is now support for extra large sized buttons. Last but not least, the new glass and glass prominent button styles bring Liquid Glass to any button in your app.

Let's move onto Sliders, which have learned a few tricks too.

They now support tick marks! The tick marks appear automatically when initializing a slider with a `step` parameter.

You can even manually place individual ticks.

Use the ticks closure to specify their location, like I'm doing here for ticks at 60% and 90%.

Sliders also let you start their track fill at a particular place. This is useful for values that may adjust left or right from a non-leading default value, like selecting faster or slower speed values on playback.

Specify the starting point with the `neutralValue` parameter. Menus across platforms have a new design and more consistent layout. Icons are consistently on the leading edge and are now used on macOS too. The same API using `Label` or standard control initializers now create the same result on both platforms.

In addition to updates to SwiftUI's controls, there are new APIs to update your controls for the new design.

Many of our controls have their corners aligned perfectly within their containers, even if the container is your iPhone! This is called corner concentricity. For example, a button that is positioned at the bottom of a sheet should share the same corner center with the corners of the sheet.

To build views that automatically maintain concentricity with their container, use the `concentric rectangle` shape. Pass the `containerConcentric` configuration to the `corner` parameter of a `rectangle` and the shape will automatically match its container across different displays and window shapes. The best way to adopt the new design is to use standard app structures, toolbars, search placements, and controls. But sometimes, your app might need a bit more customization. Next, I'll share how to build custom Liquid Glass elements for your app.

Maps is a great example for this use-case with their custom glass controls that gracefully float above the map content.

In a similar fashion, I'm going to add badges to the Landmarks app for each landmark people visit. Let's start by creating a custom badge view with the Liquid Glass effect! To add glass to your custom views, use the `glassEffect` modifier. By default, a glass effect will be applied within a capsule shape behind your content.

SwiftUI automatically uses a vibrant text color that adapts to maintain legibility against colorful backgrounds.

Customize the shape of the glass effect by providing a shape to the modifier.

For especially important views, use a tint modifier.

Similar to toolbar buttons, only use this to convey meaning and not just for visual effect.

And just like text within a glass effect, the tint also uses a vibrant color that adapts to the content behind it. On iOS, for custom controls or for containers with interactive elements, add the `interactive` modifier to the glass effect. Glass reacts to user interaction by scaling, bouncing, and shimmering, matching the effect provided by toolbar buttons and sliders. Now that we have our custom badge, let's bring multiple badges together so they interact and blend with each other. To combine multiple glass elements, use the `GlassEffectContainer`. This grouping is essential for visual correctness. The glass material reflects and refracts lights, picking colors from nearby content.

This effect is achieved by sampling content from an area larger than itself.

However, glass can not sample other glass, so having nearby glass elements in different containers will result in inconsistent behavior.

Using a glass container allows these elements to share their sampling region, providing a consistent visual result.

In the Landmarks app, I am using the `GlassEffectContainer` to group my badges. When expanding my badges, I get this wonderful fluid morphing! Add these transitions to your own glass container by using the `glassEffectID` modifier.

To configure this, I first declare a local namespace. Then, I associate the namespace with each of the `glassEffect` elements in my expanded stack of badges and with my toolbar button. Now, when I tap the button again, the award badges are re-absorbed gracefully! The liquid glass effect offers an excellent way to highlight the functionality that makes your app truly unique.

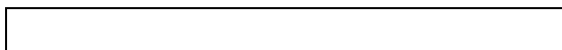
I hope you enjoyed this quick tour of applying the new design and using Liquid Glass. Now it's your turn! Adopt the new design in your app by building it with Xcode 26. I think you'll appreciate how much you get automatically with standard controls.

Audit the flow of your app and identify whether any views need changes, paying special attention to background colors behind sheets and toolbars that you can remove. Finally, build expressive components with Liquid Glass that truly make your app stand out. I hope you have a brilliant time playing with the new design! Keep on shining!

-
- ◦ 0:00 - [Introduction](#)
- ◦ iOS 26 and macOS Tahoe introduce Liquid Glass, a new adaptive material for controls and navigational elements inspired by glass and liquid. This material changes color as someone scrolls and makes controls like toggles and sliders more interactive and visually appealing. The updates apply across all Apple platforms and include changes to structural components like `TabView` and `NavigationSplitView`, toolbars, search experiences, and the introduction of new APIs for developers to customize their apps' UI. The Landmarks app demonstrates these new features.
- ◦ 3:07 - [App structure](#)
- ◦ App structure is updated with refined APIs, including `NavigationSplitView`, `TabView`, and `Sheets`, all featuring the new Liquid Glass design. `NavigationSplitView` now has a floating Liquid Glass sidebar with a 'backgroundExtensionEffect' modifier that allows views to extend outside the safe area without clipping. `TabViews` on iPhone have a floating tab bar that can minimize on scroll, enhancing content visibility, and can include a bottom accessory view for additional controls. `Sheets` now have a Liquid Glass background that transitions smoothly between partial and full heights. They can also morph out of presenting buttons, creating a seamless user experience. Other presentations like menus, alerts, and popovers also flow smoothly from Liquid Glass controls. Dialogs now automatically morph out of their presenting buttons.
- ◦ 7:49 - [Toolbars](#)
- ◦ The new app design features floating toolbars on Liquid Glass surfaces with automatic item grouping. You can use the `ToolbarSpacer` API to create custom

grouping and spacing, enhancing visual clarity. Add badges with a single line of code to indicate notifications. Monochrome icons reduce noise and an automatic scroll edge effect blurs content under toolbars for legibility. You can remove extra backgrounds and adjust for denser UIs.

- 11:10 - [Search](#)
- The new design introduces two main search patterns across platforms: Toolbar Search. The search field is at the bottom of the screen on iPhone, top-trailing on iPad/Mac, or minimized to a button which expands when tapped. Dedicated Search Page. Search is accessed via a tab, replacing the tab bar with a search field and app content. The search field appears centered above suggestions on iPad and Mac.
- 14:03 - [Controls](#)
- The new design introduces a consistent family resemblance across platforms for various controls. Buttons now have a capsule shape by default, with taller heights on macOS for better readability and larger click targets. Extra-large buttons and new glass styles are also available. Sliders now support tick marks, manual tick placement, and starting track fills from specific values. Menus have a fresh design with consistent icon placement across platforms. The new design also includes corner concentricity, ensuring controls align perfectly within their containers.
- 17:57 - [Liquid Glass effects](#)
- To customize an app further, you can build custom Liquid Glass elements. For example, the Landmarks app uses badges with the Liquid Glass effect. To achieve this, apply the 'glassEffect' modifier to custom views, which you can shape and tint for emphasis. The 'interactive' modifier is now available for iOS controls to enable scaling, bouncing, and shimmering upon interaction. Group multiple glass elements within a 'GlassEffectContainer' to ensure consistent visual results because glass cannot sample other glass. Create transitions between glass elements by using the 'glassEffectID' modifier, which allows for fluid morphing effects. This technique enhances an app's visual appeal and uniqueness, making it stand out.
- 21:31 - [Next steps](#)
- The new design in Xcode 26 includes standard controls that provide automatic enhancements and Liquid Glass for creating expressive components. Audit your app flows, remove unnecessary background colors, and leverage Liquid Glass to make your apps stand out.



Developer Footer

- [Videos](#)
- [WWDC25](#)
- Build a SwiftUI app with the new design



Platforms

[Open Menu](#) [Close Menu](#)

- [iOS](#)
- [iPadOS](#)
- [macOS](#)
- [tvOS](#)
- [visionOS](#)
- [watchOS](#)
- [App Store](#)



Tools

[Open Menu](#) [Close Menu](#)

- [Swift](#)
- [SwiftUI](#)
- [Swift Playground](#)
- [TestFlight](#)
- [Xcode](#)
- [Xcode Cloud](#)
- [Icon Composer](#)
- [SF Symbols](#)



Technologies

[Open Menu](#) [Close Menu](#)

- [Accessibility](#)
- [Accessories](#)
- [Apple Intelligence](#)
- [Audio & Video](#)
- [Augmented Reality](#)
- [Business](#)
- [Design](#)
- [Distribution](#)
- [Education](#)
- [Games](#)
- [Health & Fitness](#)
- [In-App Purchase](#)
- [Localization](#)
- [Maps & Location](#)
- [Machine Learning & AI](#)
- [Security](#)
- [Safari & Web](#)



Resources

[Open Menu](#) [Close Menu](#)

- [Documentation](#)
- [Downloads](#)
- [Sample Code](#)
- [Videos](#)



Support

[Open Menu](#) [Close Menu](#)

- [Help Guides & Articles](#)
- [Contact Us](#)
- [Forums](#)
- [Feedback & Bug Reporting](#)
- [System Status](#)



Account

[Open Menu](#) [Close Menu](#)

- [Apple Developer](#)
- [App Store Connect](#)
- [Certificates, IDs, & Profiles](#)
- [Feedback Assistant](#)



Programs

[Open Menu](#) [Close Menu](#)

- [Apple Developer Program](#)
- [Apple Developer Enterprise Program](#)
- [App Store Small Business Program](#)
- [MFi Program](#)
- [Mini Apps Partner Program](#)
- [News Partner Program](#)
- [Video Partner Program](#)
- [Security Bounty Program](#)
- [Security Research Device Program](#)



Events

[Open Menu](#) [Close Menu](#)

- [Meet with Apple](#)
- [Apple Developer Centers](#)
- [App Store Awards](#)
- [Apple Design Awards](#)
- [Apple Developer Academies](#)
- [WWDC](#)

Read the [latest news](#) Get the [Apple Developer app](#).



Light



Dark



Auto

▼

Copyright © 2026 [Apple Inc.](#) All rights reserved.
[Terms of Use](#) [Privacy Policy](#) [Agreements and Guidelines](#)